The Linearity Ladder: Where Erasure and Duplication Enter a Two-Combinator Arithmetic

Working paper v0.1, Project secret3, August 11, 2026

Abstract

In the calculus over K and S, every K firing erases a subterm and every S firing duplicates one. The terms that renounce these powers form the classical linear and affine fragments, BCI and BCK logic. This paper asks where erasure and duplication enter the arithmetic developed in the companion papers, and answers with compiler-checked accounting. Two primitive laws price every firing: a K step removes exactly the occurrences of its discarded argument, an S step adds exactly the occurrences of its shared argument. The rewrite system R_arith then splits along the same axis, and exactly: the successor rule of multiplication duplicates the multiplicand, the zero rule of multiplication erases it, and both addition rules are linear, parameter for parameter. At the level of whole derivations, the canonical unfolding of addition is multiplication-free, and the canonical multiplication of m by k fires exactly m + 1 multiplicative rules, its only non-linear firings. The finding in one sentence: in this arithmetic, multiplication is where erasure and duplication enter number, and addition touches neither. All statements are theorems in a total Idris2 module.

1. Introduction

The two combinators act asymmetrically on their arguments. K x y reduces to x and discards y; S x y z reduces to x z (y z) and hands z to two futures at once. Renouncing one or both powers yields the classical ladder of fragments: the linear terms (BCI, every argument used exactly once), the affine terms (BCK, at most once), and the full calculus [3, 4, 5]. The ladder is old. What this paper locates is its exact position inside a specific arithmetic: the system R_arith of [1], four rules that carry addition and multiplication on frozen boxes inside the weak normal forms of the calculus, machine-proved sound and studied further in [2].

The contributions are four compiled theorem groups. First, the primitive accounting: occurrence laws for the K redex and the S redex that make every firing an exact ledger entry (Section 2). Second, the rule split: the four rules of R_arith fall on the ladder exactly, with the two multiplication rules carrying the duplicating and the erasing face and the two addition rules exactly linear (Section 3). Third, derivation-level counts: addition derivations fire no multiplicative rule at all, and the canonical derivation of m times k fires exactly m + 1 of them (Section 4). Fourth, the classification table for the project's combinators and numerals, including the smallest witness that linear input-output behavior can ride on non-linear internals (Section 5).

Everything here is plain rewriting arithmetic. The statements are types, the proofs are total terms, and idris2 —check is the verification; Appendix B reproduces the relevant module section verbatim.

2. Two primitive laws

Occurrences are counted by a three-line function over the term syntax: occ x (Var y) is one if the names match and zero otherwise, the constants count zero, and applications add. The two contraction schemes then obey exact laws, both compiled.

```
kStepOcc : (x : String) -> (a, b : Term) ->
  occ x (App (App CK a) b) = occ x a + occ x b
```

```
sStepOcc : (x : String) -> (a, b, c : Term) ->
  occ x (App (App a c) (App b c)) =
  occ x (App (App (App CS a) b) c) + occ x c
```

Read as a ledger: a K firing takes a redex with occ x a + occ x b occurrences to a result with occ x a, removing exactly the occurrences of the discarded argument. An S firing takes its redex to a result with exactly occ x c more, the occurrences of the shared argument, counted once extra for the second future. The first law is definitional; the second is two lines of additive regrouping. Nothing else in the calculus creates or destroys an occurrence: the context closure of weak reduction is occurrence-neutral by construction.

3. The rule set splits

R_arith consists of four rules [1]. R1 unfolds multiplication by a successor, R2 multiplication by zero, R3 addition by a successor, R4 addition by zero. Each rule has parameter slots, and the question is what each rule does to the occurrences in each slot. The compiled answer:

```
r1Occ : occ x (r1R u v) = occ x (r1L u v) + occ x v
r2Occ : occ x (r2L v)  = occ x ki + occ x v
r3Occ : occ x (r3R u v) = occ x (r3L u v)
r4Occ : occ x v = occ x (r4L v)
```

R1 adds exactly the occurrences of v: the multiplicand is duplicated, once for the residual product and once for the sum that absorbs it. R2 is the mirror: its right side is the zero numeral, occ x ki = 0, so the rule erases exactly the occurrences of v. R3 and R4 are equalities with no correction term: both addition rules are exactly linear, every parameter occurrence preserved. Three of the four theorems are definitional (Refl); R1 costs one commutativity. Concrete witnesses ride along: under R1 the count of the multiplicand jumps from one to two, under R2 from one to zero.

This is the paper's central observation stated as compiled mathematics. The Peano recursions, oriented as rewrite rules inside the two-combinator syntax, distribute the powers of the calculus in the most legible way possible: multiplication owns both the S-face and the K-face, addition owns neither.

4. Derivation-level accounting

Occurrence laws price single firings. The next level counts firings across whole derivations. A one-line tag assigns one to each R1 or R2 firing and zero to R3, R4, and context steps; usesMul sums the tags along a derivation. Two exact counts follow.

```
usesMulSum : (k : Nat) -> (x : Term) ->
  usesMul (sumSteps k x) = 0

usesMulBox : (m, k : Nat) ->
  usesMul (boxReduces m k) = S m
```

The first says the canonical unfolding of a sum box is multiplication-free: addition computes entirely inside the linear rule fragment. The second gives the canonical multiplication derivation its exact non-linear budget: reducing the product box of m and k to the numeral of m times k fires the duplicating rule m times and the erasing rule once, m + 1 multiplicative firings in total, and every other step in the derivation is a linear addition rule or a context move. Duplication does the work for as long as there is a successor to unfold; erasure closes the account at zero. The proofs are struc-

tural inductions over the derivations of [1] and [2], with three small distribution lemmas for the counter.

5. The ladder census

The project's combinators and standard forms classify as follows. Actions cited are machine-checked in the shared module [1, 2].

Form	Action	Class
I = S K K	x maps to x	linear
B	f, g, x map to f (g x)	linear
K	discards its second argument	affine, not linear
S	shares its third argument	duplicating
zero numeral K I	erases its iterator f	affine, not linear
one numeral	f, x map to f x	linear input-output
numeral of n, n at least 2	f appears n times in the result	duplicating
R3, R4 (addition)	every parameter once	linear
R1 (multiplication, successor)	multiplicand twice	duplicating
R2 (multiplication, zero)	multiplicand zero times	erasing

One row deserves its footnote. The numeral of one has linear input-output behavior: applied to f and x it reduces to f x, each argument once. Its internals are not linear: the reduction passes through the zero numeral, which fires a K and erases an f along the way. This is the smallest witness in the project that static accounting (occurrences in the normal form) and dynamic accounting (firings along the reduction) are different ledgers. The occurrence laws of Section 2 price the dynamic ledger per firing; totalizing it over the arithmetic, and pricing it thermodynamically, is the entropy-budget experiment queued in the program note [8].

6. Relation to the fragments, and scope

The linear and affine combinatory fragments are classical objects, BCI and BCK logic, with a standard literature [3, 4, 5]. This paper claims no novelty for the ladder. The finding is its exact location inside a specific machine-proved arithmetic: which rules sit on which rung, with exact occurrence corrections, and how the non-linear budget of a whole multiplication derivation closes in m + 1. The accounting is rule-level and derivation-level; no type system for the fragments is built here, and no claim is made beyond the rule set and the canonical derivations studied. The physical reading of the K column, each firing an erasure event in Landauer's sense [6], is deliberately left as a pointer: the counting above is exact and belongs to this paper, the joules belong to the experiment that will spend them.

7. Conclusion

The rewrite arithmetic of the companion papers splits along the oldest axis the calculus has. Both addition rules are exactly linear. Both multiplication rules are exactly non-linear, one duplicating and one erasing, and the canonical multiplication of m by k spends its non-linearity in exactly m + 1 firings. In this arithmetic, multiplication is where erasure and duplication enter number, and addition touches neither. The next experiments of the program continue outward from here: the combinator soup and the entropy budget [8].

Appendix A: Methods

All theorems live in the shared proof module Proofs.idr of Projects secret1 and secret2, which stands at 1,052 lines after this paper's additions, is declared %default total, and contains no postulates and no holes; the single command idris2 —check Proofs.idr is the entire verification, and the entry point prints the theorem inventory. The full module is reproduced in the companion papers [1, 2]; Appendix B reproduces the section added by this paper verbatim. One engineering note: the derivation counter must compute through an endpoint transport inside the multiplication derivation, so the transport was moved into a named helper (sumStepsNum) and discharged by a one-line lemma (usesMulRepl) that reduces the counter across replace.

Appendix B: The linearity ladder section of the proof module

```
--------------------------------------------------------------------
-- Tier 2, part E: the linearity ladder (E8)
--------------------------------------------------------------------

||| occurrences of a variable in a term
occ : String -> Term -> Nat
occ x (Var y)   = if x == y then 1 else 0
occ x CK        = 0
occ x CS        = 0
occ x (App a b) = occ x a + occ x b

||| primitive accounting: every K firing removes the occurrences of
||| the discarded argument, every S firing adds the occurrences of
||| the shared argument
kStepOcc : (x : String) -> (a, b : Term) ->
  occ x (App (App CK a) b) = occ x a + occ x b
kStepOcc x a b = Refl

sStepOcc : (x : String) -> (a, b, c : Term) ->
  occ x (App (App a c) (App b c)) =
  occ x (App (App (App CS a) b) c) + occ x c
sStepOcc x a b c =
  trans (plusSwapMid (occ x a) (occ x c) (occ x b) (occ x c))
        (plusAssociative (plus (occ x a) (occ x b)) (occ x c) (occ x c))

||| the rule set splits: R1 duplicates its second parameter,
||| R2 erases it, R3 and R4 are exactly linear
r1Occ : (x : String) -> (u, v : Term) ->
  occ x (Proofs.r1R u v) = occ x (Proofs.r1L u v) + occ x v
r1Occ x u v = plusCommutative (occ x v) (plus (occ x u) (occ x v))

r2Occ : (x : String) -> (v : Term) ->
  occ x (Proofs.r2L v) = occ x Proofs.ki + occ x v
r2Occ x v = Refl

r3Occ : (x : String) -> (u, v : Term) ->
  occ x (Proofs.r3R u v) = occ x (Proofs.r3L u v)
r3Occ x u v = Refl

r4Occ : (x : String) -> (v : Term) ->
  occ x v = occ x (Proofs.r4L v)
r4Occ x v = Refl
```

```
||| concrete witnesses
r1DupWitness : Proofs.occ "v" (Proofs.r1R (Var "u") (Var "v")) = 2
r1DupWitness = Refl

r2EraseWitness : Proofs.occ "v" Proofs.ki = 0
r2EraseWitness = Refl

zeroErasesIter : Proofs.occ "f" (Proofs.fpow 0 (Var "x")) = 0
zeroErasesIter = Refl

||| multiplicative firings in an R derivation: R1 and R2 count,
||| the additive rules and context closure do not
mulTag : StepR t u -> Nat
mulTag (RuleR1 _ _) = 1
mulTag (RuleR2 _)   = 1
mulTag (RuleR3 _ _) = 0
mulTag (RuleR4 _)   = 0
mulTag (RCtxL p)    = mulTag p
mulTag (RCtxR p)    = mulTag p

usesMul : StepsR t u -> Nat
usesMul RDone        = 0
usesMul (RThen s ss) = mulTag s + usesMul ss

usesMulTrans : (xs : StepsR a b) -> (ys : StepsR b c) ->
  usesMul (rStepsTrans xs ys) = usesMul xs + usesMul ys
usesMulTrans RDone ys = Refl
usesMulTrans (RThen s xs) ys =
  trans (cong (plus (mulTag s)) (usesMulTrans xs ys))
        (plusAssociative (mulTag s) (usesMul xs) (usesMul ys))

usesMulAppRs : {l, r, r' : Term} -> (ss : StepsR r r') ->
  usesMul (rAppRs {l = l} ss) = usesMul ss
usesMulAppRs RDone        = Refl
usesMulAppRs (RThen s ss) = cong (plus (mulTag s)) (usesMulAppRs {l = l} ss)

usesMulRepl : (e : t = t') -> (d : StepsR a t) ->
  usesMul (replace {p = StepsR a} e d) = usesMul d
usesMulRepl Refl d = Refl

||| addition is multiplication-free
usesMulSum : (k : Nat) -> (x : Term) -> usesMul (sumSteps k x) = 0
usesMulSum Z     x = Refl
usesMulSum (S k) x = trans (usesMulAppRs {l = Proofs.sb} (sumSteps k x))
(usesMulSum k x)

usesMulSumNum : (k, j : Nat) -> usesMul (sumStepsNum k j) = 0
usesMulSumNum k j =
  trans (usesMulRepl (sbPowNum k j) (sumSteps k (num j)))
        (usesMulSum k (num j))

||| the canonical multiplication derivation fires exactly m + 1
||| multiplicative rules, and they are its only non-linear firings
usesMulBox : (m, k : Nat) -> usesMul (boxReduces m k) = S m
```

```
usesMulBox Z     k = Refl
usesMulBox (S m) k =
  cong S $
  trans (usesMulTrans (rAppRs (boxReduces m k)) (sumStepsNum k (m * k))) $
  trans (cong2 plus (usesMulAppRs {l = App CS (App Proofs.skb (Proofs.num k))}
(boxReduces m k)) (usesMulSumNum k (m * k))) $
  trans (plusZeroRightNeutral (usesMul (boxReduces m k))) $
  usesMulBox m k
```

References

1. Project secret1, ‘A Tree-Based Number System over the Combinators K and S, and Primality as Unreachability’, working paper v0.4, August 11, 2026.
2. Project secret2, ‘Transparent Arithmetic over the Combinators K and S: Divisibility Automata, Factoring as Rewriting, and the Search for a Prime-Revealing Representation’, working paper v0.8, August 11, 2026.
3. K. Bimbó, *Combinatory Logic: Pure, Applied and Typed*, CRC Press, 2012.
4. J.-Y. Girard, ‘Linear logic’, Theoretical Computer Science 50 (1987), 1–101.
5. H. B. Curry and R. Feys, *Combinatory Logic*, Vol. I, North-Holland, 1958.
6. R. Landauer, ‘Irreversibility and heat generation in the computing process’, IBM Journal of Research and Development 5 (1961), 183–191.
7. E. Brady, ‘Idris 2: Quantitative Type Theory in Practice’, *Proceedings of ECOOP 2021*, LIPIcs 194, 2021.
8. Project secret3, ‘The Reach of K and S: A Program Note on Demonstrations in Mathematics and Physics’, program note v0.2, August 11, 2026.